ORM & Hibernate

From JDBC → ORM → Hibernate → JPA

Goal: understand *why* ORM was invented, what Hibernate actually is, then get hands-on with Hibernate and JPA.

Today's Roadmap

Topic	Format
JDBC recap and its pain points	Short lecture
What is ORM? What is Hibernate? Other implementations	Lecture (concepts only)
Hibernate fundamentals	Lecture + hands-on lab
From Hibernate to JPA	Lecture
JPA Configuration & Hello JPA	Lecture + hands-on lab

We start with **concepts only** — no code yet. Code only shows up once the ideas behind it are clear.

JDBC Recap & Its Pain Points

What is JDBC?

- Stands for **J**ava **D**ata**b**ase **C**onnectivity
- A set of Java APIs for accessing relational databases from a Java program
- A Java API that enables Java programs to execute SQL statements and interact with any SQL-compliant database
- Makes it possible to write a single database application that can run on different platforms and interact with different DBMS engines

---

Pros and Cons of JDBC

Pros

- Simple, direct control over SQL and the database
- Works with virtually any relational database
- No extra framework required — it's part of the JDK

Cons

- Large programming overhead in bigger projects
- Transactions and concurrency must be hand-coded
- Handling and closing connections is a constant, manual concern
- No encapsulation — SQL and object logic are tangled together
- Hard to implement clean, layered (MVC-style) design on top of it

---

Why ORM? (The Problem Area)

When working with object-oriented systems, there is a fundamental **mismatch** between the object model and the relational database:

- Objects have **identity**, **inheritance**, and **graphs of references** to other objects
- Relational tables only have **rows**, **foreign keys**, and **flat columns**
- A Java object naturally *contains* other objects (e.g. an Employee that holds a Department); a database table can only *point* to another table via a foreign key
- Someone has to bridge that gap — and with plain JDBC, **you** are that someone, by hand, every time

---

The Preferred Solution

Use an **Object-Relational Mapping (ORM) system**, sitting between your Java objects and the database:

```
Your Java Objects  ⇄  [ ORM — "magic happens here" ]  ⇄  Database
```

- Provides a simple API for storing and retrieving Java objects directly to and from the database

---

What is ORM?

Object-Relational Mapping (ORM) is a programming technique for converting data between relational databases and object-oriented programming languages.

- Lets business code access **objects**, rather than database tables
- Hides the details of SQL queries from your object-oriented logic
- Is **based on JDBC under the hood** — it doesn't replace JDBC, it automates it

---

ORM Pros

- No need to deal with the database implementation directly — little need to write SQL statements
- Entities are modeled around **business concepts**, rather than database structure
- Faster application development
- Transaction management and automatic key generation are handled for you

---

Java ORM / Persistent Frameworks

There are more than 23 ORM implementations for Java. Some of the most common:

- **Hibernate** — open-source ORM framework, the most widely used
- **Java Persistence API (JPA)** — the standard specification
- **TopLink** — by Oracle
- **EclipseLink** — the Eclipse persistence platform
- **iBATIS (MyBatis)** — maintained by the Apache Software Foundation
- **Enterprise Objects Framework** — Mac OS X/Java, part of Apple WebObjects

---

What is Hibernate?

Hibernate is a free, open-source **Object-Relational Mapping framework for Java** — the most widely adopted ORM in the Java ecosystem.

- Maps Java classes to database tables, and Java data types to SQL data types, automatically
- Predates JPA, and later became the most popular **implementation** of the JPA specification, while still offering its own native API
- Takes care of generating SQL for you, and converting the results back into Java objects
- Manages the details you'd otherwise hand-code with JDBC: connections, transactions, and caching
- Works against virtually any relational database, by simply changing its configured "dialect"

Why Hibernate, Specifically?

- Eliminates the repetitive JDBC boilerplate — no manual `ResultSet` mapping, no hand-built SQL strings
- Automatically tracks changes to your objects and generates the right `INSERT` / `UPDATE` / `DELETE` statements for you (**dirty checking**)
- Built-in caching reduces unnecessary trips to the database

- Understands relationships between entities (one-to-many, many-to-many, etc.) declaratively, instead of manual joins
- Mature, battle-tested, huge community and documentation — a safe default choice for Java persistence

---

Hibernate Fundamentals

Hibernate Architecture

```
Configuration → SessionFactory → Session → Transaction
 (read once,     (heavyweight,     (lightweight,
  at startup)     built ONCE        one per unit
                  per app)          of work)
```

Component	Role	Lifetime
`Configuration`	Reads your settings (DB URL, dialect, mapped classes) and builds the factory	Used once, at startup
`SessionFactory`	A thread-safe factory that produces `Session` objects; expensive to create	**One per application**
`Session`	Represents a single unit of work against the DB; wraps a JDBC `Connection`	**One per request/transaction** — cheap to open and close
`Transaction`	A single atomic unit of work (commit or rollback together)	Scoped to one `Session` usage

> Mental model: **`Session` ≈ a JDBC `Connection`, but object-aware.**

Hibernate / JPA Entity Annotations

> ⓘ **Context** Same entity class used by both a Hibernate-native demo and a JPA demo. **`@Entity`**, **`@Id`**, **`@Column`**, etc. are part of the JPA specification itself — the same mapping works whether accessed through a Hibernate **`Session`** or a standard JPA **`EntityManager`**.

Reference Entity

```java
package com.training.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
```

```java
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private int id;

    @Column(name = "name")
    private String name;

    @Column(name = "department")
    private String department;

    @Column(name = "salary")
    private double salary;

    public Employee() {}

    public Employee(String name, String department, double salary) {
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    // getters/setters omitted for brevity
}
```

@Entity

📋 **Purpose** Marks a class as a JPA entity — maps it to a database table.

```
@Entity
```

Parameter	Type	Default	Description
`name`	String	simple class name	Logical name used in **JPQL** queries (`SELECT e FROM Employee e`)

```java
@Entity(name = "Emp") // JPQL must now reference "Emp"
```

⚠ **Gotcha** `@Entity(name=...)` sets the JPQL entity name, NOT the table name. Table name is set via `@Table`. Students frequently confuse these two.

@Table

> 📋 **Purpose** Specifies database table details. Optional — defaults to the entity/class name if omitted.

```
@Table(name = "employee")
```

Parameter	Type	Default	Description
`name`	String	entity name	Table name in the database
`catalog`	String	`""`	Catalog the table belongs to
`schema`	String	`""`	Schema the table belongs to
`uniqueConstraints`	`UniqueConstraint[]`	`{}`	Composite unique constraints across multiple columns
`indexes`	`Index[]`	`{}`	Indexes to create on the table

```
@Table(
    name = "employee",
    schema = "hr",
    uniqueConstraints = @UniqueConstraint(columnNames = {"name", "department"}),
    indexes = @Index(name = "idx_dept", columnList = "department")
)
```

`@Id`

> 📋 **Purpose** Marks the field as the primary key. Every entity must have exactly one (or a composite key via `@EmbeddedId` / `@IdClass`).

```
@Id
```

No parameters — just marks the field/property.

`@GeneratedValue`

> 📋 **Purpose** Tells Hibernate how to auto-generate the primary key value.

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

Parameter	Type	Default	Description
`strategy`	`GenerationType`	`AUTO`	The generation strategy (see below)
`generator`	String	`""`	Name of a custom generator (references a named `@SequenceGenerator` / `@TableGenerator`)

`GenerationType` values

> 🔥 **Worth real slide time — this is where most interview questions live.**

Strategy	How it works	Notes
`IDENTITY`	DB auto-increment column generates the ID	Uses native `AUTO_INCREMENT` (MySQL) / `IDENTITY` (SQL Server). **Cannot be batched** — Hibernate must insert immediately to retrieve the ID, hurting batch-insert performance
`SEQUENCE`	Uses a DB sequence object	Needs `@SequenceGenerator`. Preferred for PostgreSQL/Oracle. Supports batching (Hibernate can pre-fetch ID ranges)
`TABLE`	Simulates a sequence via a separate counter table	Portable across all DBs but slowest — requires its own transaction to increment
`AUTO`	Hibernate picks a strategy based on DB dialect	Good default for portability; can surprise you across different DBs
`UUID` *(JPA 3.1+ / Hibernate 6)*	Generates a UUID	No DB round-trip needed to get the ID

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "emp_seq")
@SequenceGenerator(
    name = "emp_seq",
    sequenceName = "employee_seq",
    initialValue = 1,
    allocationSize = 1
)
private int id;
```

`@SequenceGenerator` parameters: `name`, `sequenceName`, `catalog`, `schema`, `initialValue` (default `1`), `allocationSize` (default `50`)

> ⚠ **Gotcha** `allocationSize` defaulting to `50` causes visible "gaps" in generated IDs if a student isn't expecting it — Hibernate reserves blocks of 50 in memory rather than hitting the DB every insert.

`@Column`

> 📋 **Purpose** Maps a field to a specific database column and its constraints. Optional — defaults to the field name if omitted.

```
@Column(name = "name")
```

Parameter	Type	Default	Description
`name`	String	field name	Column name in the DB
`nullable`	boolean	`true`	Adds `NOT NULL` if `false` — **DDL generation only, not runtime validation**
`unique`	boolean	`false`	Adds a unique constraint on this column
`insertable`	boolean	`true`	Whether included in INSERT statements
`updatable`	boolean	`true`	Whether included in UPDATE statements
`length`	int	`255`	Column length — for `String` / `varchar` types
`precision`	int	`0`	Total digits for `BigDecimal` /numeric types
`scale`	int	`0`	Digits after decimal point for numeric types
`columnDefinition`	String	`""`	Raw SQL fragment for the column definition (e.g. `"TEXT"`, `"DECIMAL(10,2)"`)
`table`	String	`""`	Which table this column belongs to (for `@SecondaryTable` mappings)

```
@Column(name = "salary", nullable = false, precision = 10, scale = 2)
private double salary;

@Column(name = "email", unique = true, length = 100)
private String email;

@Column(name = "created_at", insertable = true, updatable = false)
private LocalDateTime createdAt; // set once, never updated
```

⚠ Gotcha `nullable = false` only affects generated DDL (the `CREATE TABLE` statement) — it does not validate at the application layer. Pair with Bean Validation (`@NotNull`) if runtime validation is needed.

Summary Table

Annotation	Required?	Purpose
`@Entity`	✅ Yes	Marks class as JPA-managed
`@Table`	❌ No (defaults to class name)	Customizes table name/schema/constraints
`@Id`	✅ Yes	Marks primary key field
`@GeneratedValue`	❌ No	Auto-generates PK value
`@Column`	❌ No (defaults to field name)	Customizes column mapping/constraints

---

CRUD via `Session`

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
```

```
// Create
Employee e = new Employee();
e.setName("Sara Ahmed");
e.setDepartment("Engineering");
e.setSalary(12000);
session.persist(e);

// Read
Employee found = session.get(Employee.class, e.getId());

// Update
found.setSalary(13000);   // no explicit "update" call needed!

// Delete
// session.remove(found);

tx.commit();
session.close();
```

🔍 Line-by-Line: CRUD via `Session`

Code	What it does
`sessionFactory.openSession()`	Opens a new unit of work; internally acquires a JDBC `Connection` from the pool
`session.beginTransaction()`	Starts a database transaction — nothing is permanently saved until `commit()`
`new Employee(); e.setX(...)`	Creates a **transient** object — plain Java, Hibernate doesn't know it exists yet
`session.persist(e)`	Makes the object **persistent** — Hibernate now tracks it and will `INSERT` it (the actual SQL fires at commit or flush time)
`session.get(Employee.class, e.getId())`	Fetches by primary key. If it's already in the first-level cache, **no SQL is even sent**
`found.setSalary(13000)`	A plain setter call — **no explicit "update" or "save" needed**. Because `found` is *persistent*, Hibernate detects the change automatically
`tx.commit()`	Hibernate compares the current state of tracked objects to what it loaded (**dirty checking**) and generates the necessary `UPDATE` / `INSERT` statements here
`session.close()`	Releases the underlying connection back to the pool; any objects loaded through this session become **detached**

> **Dirty checking** is the biggest mental shift from JDBC: you never write `UPDATE` statements by hand again — you mutate the object, and Hibernate figures out the diff.

🧪 Hands-On Lab 1

Implement a basic Hibernate `findEmployeeById` lookup.

1. Map `Employee` with `@Entity`, `@Id`, `@GeneratedValue`, `@Column` (as shown above)
2. Configure a `SessionFactory` (via `hibernate.cfg.xml` or programmatically)
3. Write `findEmployeeById(int id)` using `session.get(Employee.class, id)`
4. Run it against a local/H2 database and confirm it returns the correct row
5. **Discuss as a group:** what would this same lookup have taken to write in raw JDBC?

Estimated time: 20–25 minutes

From Hibernate to JPA

JPA vs. Hibernate

JPA	Hibernate
A **specification** — a set of interfaces and rules defined by Jakarta EE	A **concrete implementation** of the JPA specification
Package: `jakarta.persistence.*`	Package: `org.hibernate.*`
Like the **JDBC API/interfaces**	Like a **specific JDBC driver**
—	Hibernate's **native API** (`Session`, `SessionFactory`)

Key idea: JPA defines *what* an ORM should provide, while Hibernate provides *how* those features are implemented.

What is JPA? Why JPA?

What: Java's standard API specification for Object-Relational Mapping — defined once, implemented by multiple vendors.

Why it matters:

- **Vendor independence** — swap Hibernate for EclipseLink or OpenJPA without rewriting your entity classes or business logic
- The **same** `@Entity`, `@Id`, `@Column` annotations you already learned are part of the JPA spec itself, not Hibernate-specific
- The **same** lifecycle states (transient/persistent/detached/removed) carry over unchanged

---

Supporting Database Management Systems

JPA/Hibernate is not tied to a single database engine. The exact same entity code can run against:

Database	Typical use
H2	In-memory, zero setup — ideal for labs and tests
MySQL / MariaDB	Common in web applications
PostgreSQL	Common in enterprise and open-source stacks
Oracle	Common in large enterprise systems
SQL Server	Common in .NET-adjacent / Microsoft shops

Switching databases is mostly a matter of changing the **dialect, driver, and connection URL** — your `@Entity` classes stay the same.

JPA Architecture & API

```
EntityManagerFactory   ≈   SessionFactory     (one per application)
EntityManager          ≈   Session            (one per unit of work)
EntityTransaction      ≈   Transaction
Persistence Context    ≈   first-level cache  (tracked entities in this EntityManager)
```

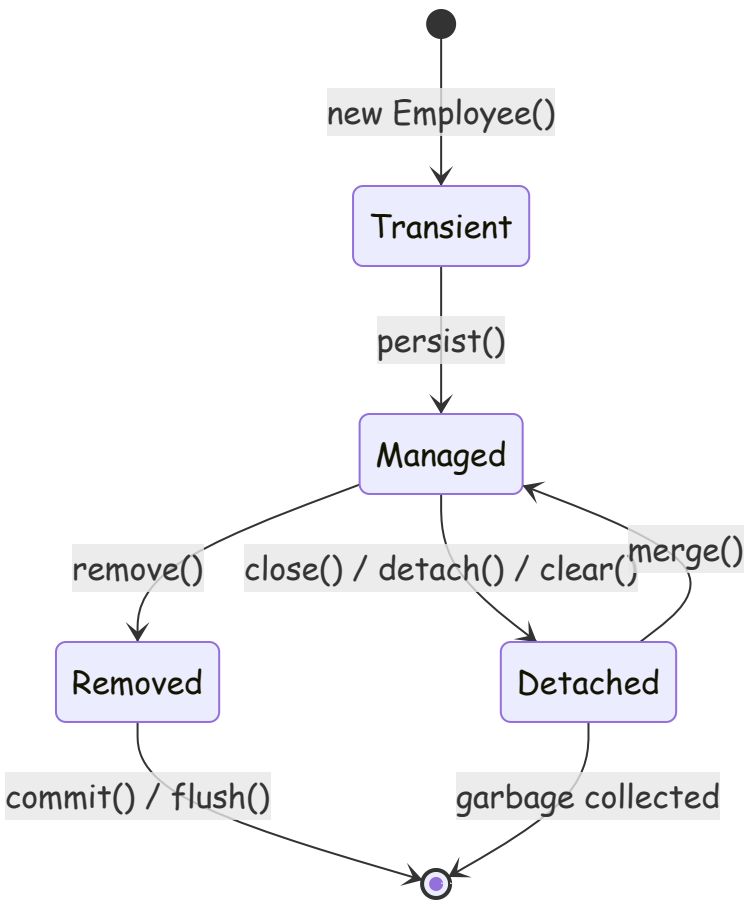
JPA Type	Role
`Persistence`	Static bootstrap class — the entry point that reads your config and builds an `EntityManagerFactory`
`EntityManagerFactory`	Thread-safe, expensive to create, built once per application
`EntityManager`	The standard equivalent of `Session` — used to `persist`, `find`, `merge`, `remove`
`EntityTransaction`	The standard equivalent of `Transaction`

Nothing from Hibernate is wasted — you are learning the same concepts under their official, portable names.

JPA Entity Lifecycle

> ⓘ **Context**
>
> Every JPA entity instance is always in exactly one of **four states** at any given moment. Which state it's in determines whether the `EntityManager` is tracking it, whether changes to it get saved, and whether it even corresponds to a database row yet. This is the single most important mental model in JPA — nearly every "why didn't my update save?" bug traces back to a misunderstanding of which state an entity was in.

The Four States



State	Also called	Meaning	Tracked by EntityManager?
Transient	New	Plain Java object — JPA has no knowledge of it	❌ No
Managed	Persistent	Attached to an open persistence context; every field change is tracked	✅ Yes
Detached	—	Was managed, but its `EntityManager` has since closed (or the entity was explicitly detached)	❌ No
Removed	—	Marked for deletion; the `DELETE` fires at commit/flush	✅ Yes (until commit)

> ⚠ **The #1 beginner bug**
>
> Mutating a **detached** entity and expecting the database to update automatically. It won't — nothing is watching it. You must call `em.merge(entity)` to reattach it (which returns a *new* managed instance) before the change will be picked up.

State Transitions — How You Get There

Transition	Method	From → To
Create	`new Employee()`	*(none)* → Transient
Make persistent	`em.persist(entity)`	Transient → Managed
Load from DB	`em.find(...)` / `em.getReference(...)`	*(none)* → Managed
Detach one entity	`em.detach(entity)`	Managed → Detached
Detach everything	`em.clear()`	Managed (all) → Detached (all)
Close persistence context	`em.close()`	Managed (all) → Detached (all)

Transition	Method	From → To
Reattach	`em.merge(entity)`	Detached → Managed *(returns a new reference!)*
Mark for deletion	`em.remove(entity)`	Managed → Removed
Re-sync from DB	`em.refresh(entity)`	Managed → Managed *(overwrites in-memory changes)*
Persist changes	`em.flush()` / commit	Managed → DB (dirty checking fires here)

Key Gotchas to Emphasize in Class

⚠ `merge()` returns a NEW object

`em.merge(detachedEntity)` does **not** reattach the object you passed in — it copies its state onto a (possibly newly loaded) managed instance and **returns that instance**. Always reassign: `entity = em.merge(entity)`. Continuing to use the old reference is a very common bug.

🔥 Flush vs. Commit

`flush()` pushes pending SQL to the database **without** ending the transaction — useful when you need a generated ID immediately, or want to run a native query that depends on unflushed changes. `commit()` flushes **and** ends the transaction.

🔥 `find()` vs `getReference()`

`em.find()` hits the DB immediately (or the cache) and returns a real object.
`em.getReference()` returns a lazy proxy without hitting the DB — accessing any field triggers the query then. Useful when you only need the entity to set up a foreign-key relationship, not to read its data.

Summary Table

From State	Action	To State
—	`new Entity()`	Transient
Transient	`persist()`	Managed
—	`find()` / `getReference()`	Managed
Managed	`detach()` / `clear()` / `close()`	Detached
Detached	`merge()`	Managed (new reference)

From State	Action	To State
Managed	`remove()`	Removed
Removed	`flush()` / commit	*(gone — row deleted)*
Managed	`refresh()`	Managed (in-memory state overwritten)

State	Meaning	How you get there
Transient	Plain Java object; Hibernate has no knowledge of it	`new Employee()`
Persistent	Attached to an open `Session`; every field change is tracked and auto-synced at commit/flush	`persist()`, or loaded via `get()` / `find()`
Detached	Was persistent, but its `Session` has since closed; changes are **no longer tracked**	`session.close()`, or `session.detach(e)`
Removed	Marked for deletion; the `DELETE` fires at commit/flush	`remove()`

First-Level (Session) Cache

```
Employee first  = session.get(Employee.class, 1);  // hits the database
Employee second = session.get(Employee.class, 1);  // NO SQL fired!

System.out.println(first == second); // true — same object reference
```

- Every `Session` keeps its **own** cache of entities it has already loaded, keyed by identifier
- Calling `get()` twice for the same ID in the same session → the second call returns the **cached object**, no SQL sent
- Cleared the moment the `Session` closes — **not** shared across sessions or across your application
- Requires **zero configuration** — always active

JPA Configuration & Hello JPA

JPA Installation / Setup

Maven dependencies needed to run JPA (via Hibernate as the provider) against an in-memory H2 database:

```
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.4.4.Final</version>
</dependency>
<dependency>
```

```
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <version>2.2.224</version>
</dependency>
```

- `hibernate-core` — provides the JPA implementation (Hibernate acting as the **JPA provider**)
- `h2` — the JDBC driver for our lab database; swap this for `mysql-connector-j`, `postgresql`, etc. in a real project

---

`persistence.xml`

```
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
             version="3.0">
  <persistence-unit name="empPU" transaction-type="RESOURCE_LOCAL">
    <class>com.training.model.Employee</class>
    <properties>
      <property name="jakarta.persistence.jdbc.url"
                value="jdbc:h2:mem:empdb"/>
      <property name="jakarta.persistence.jdbc.driver"
                value="org.h2.Driver"/>
      <property name="hibernate.hbm2ddl.auto" value="update"/>
      <property name="hibernate.show_sql" value="true"/>
    </properties>
  </persistence-unit>
</persistence>
```

🔍 Line-by-Line: `persistence.xml`

Element / Attribute	Meaning
`<persistence-unit name="empPU" ...>`	Names this configuration `empPU` — this exact string is what you pass to `Persistence.createEntityManagerFactory(...)` in code
`transaction-type="RESOURCE_LOCAL"`	We manage transactions ourselves (`em.getTransaction().begin()/commit()`), as opposed to `JTA`, which is managed by an application server
`<class>com.training.model.Employee</class>`	Explicitly registers which entity classes belong to this persistence unit
`jakarta.persistence.jdbc.url`	The standard JPA property name for the JDBC connection string
`jakarta.persistence.jdbc.driver`	The JDBC driver class to load
`hibernate.hbm2ddl.auto = update`	A Hibernate-specific property telling it to auto-create/update the schema to match your entities — convenient for labs, **dangerous in production**
`hibernate.show_sql = true`	Prints every generated SQL statement to the console — useful while learning, so you can see exactly what Hibernate is doing behind the `EntityManager` calls

Configuration Properties — Common Ones

Property	Purpose
`jakarta.persistence.jdbc.url`	Database connection string
`jakarta.persistence.jdbc.user` / `.password`	Database credentials
`hibernate.hbm2ddl.auto`	Schema strategy: `create`, `create-drop`, `update`, `validate`, `none`
`hibernate.show_sql`	Log generated SQL to console
`hibernate.format_sql`	Pretty-print the logged SQL
`hibernate.dialect`	The SQL dialect Hibernate should generate (usually auto-detected in modern versions)

> `hbm2ddl.auto=validate` (or `none`, with migrations handled by a tool like Flyway/Liquibase) is the production-safe choice — `update` and `create` are for learning and local development only.

Mapping Files vs. Annotations

(Development Strategies)

Strategy	Description	When to use
Annotation-based	`@Entity`, `@Column`, etc. placed directly on the class — what we used all day today	Default for modern projects; mapping lives right next to the code it describes
XML mapping files	`orm.xml` (JPA) or `.hbm.xml` (Hibernate-specific) — mapping defined entirely outside the class	Useful when you cannot modify the entity source, or for very large legacy configurations that predate annotations

Both strategies produce **identical runtime behavior** — this is purely a matter of where the mapping information lives. This course uses annotations going forward.

First Example — Hello JPA

```
EntityManagerFactory emf =
        Persistence.createEntityManagerFactory("empPU");
EntityManager em = emf.createEntityManager();

em.getTransaction().begin();

Employee e = new Employee();
e.setName("Omar Khaled");
```

```
e.setDepartment("QA");
e.setSalary(9500);
em.persist(e);

em.getTransaction().commit();

Employee found = em.find(Employee.class, e.getId());
System.out.println(found.getName());

em.close();
emf.close();
```

🔍 Line-by-Line: Hello JPA

Code	What it does
`Persistence.createEntityManagerFactory("empPU")`	Reads `persistence.xml`, finds the unit named `"empPU"`, and builds the factory — this string **must match** the `name` attribute exactly
`emf.createEntityManager()`	Opens an `EntityManager` — the standard-API equivalent of `Session`
`em.getTransaction().begin()`	Starts a transaction, exactly like `session.beginTransaction()`
`new Employee(); e.setX(...)`	Same as before — a **transient** object
`em.persist(e)`	Standard-API equivalent of `session.persist(e)` — makes the object **persistent**
`em.getTransaction().commit()`	Flushes changes and commits — dirty checking and SQL generation happen here
`em.find(Employee.class, e.getId())`	Standard-API equivalent of `session.get(...)`
`em.close()` / `emf.close()`	Releases resources — mirrors `session.close()`

> Notice this is **almost line-for-line identical** to the Hibernate native code — same concepts, same order of operations, different (standardized) type names.

Recap — Day 1

- JDBC gives you rows, not objects — the impedance mismatch is real
- ORM solves it: describe the mapping once, reuse it everywhere
- Hibernate is the most widely used Java ORM — open-source, implements JPA, and also has its own native API
- Other implementations exist: JPA (the standard), TopLink, EclipseLink, MyBatis, and more
- Hibernate's native flow: `Configuration → SessionFactory → Session → Transaction`
- **Dirty checking**: mutate the object, Hibernate writes the `UPDATE` for you
- Entity lifecycle: **transient** → **persistent** → **detached** → **removed**
- JPA is the **standardized specification** on top of what Hibernate already does

- `EntityManagerFactory` / `EntityManager` map directly onto `SessionFactory` / `Session`
